

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

Лабораторная работа №6  
По дисциплине: «ОМО»  
Тема: "Рекуррентные нейронные сети"

Выполнил:  
Студент 3-го курса  
Группы АС-66  
Осовец А.О.  
Проверил:  
Крощенко А.А.

Брест 2025

## Вариант 7

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию. Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

| № варианта | a   | b   | c    | d   | Кол-во входов ИНС | Кол-во НЭ в скрытом слое |
|------------|-----|-----|------|-----|-------------------|--------------------------|
| 1          | 0.1 | 0.1 | 0.05 | 0.1 | 6                 | 2                        |
| 2          | 0.2 | 0.2 | 0.06 | 0.2 | 8                 | 3                        |
| 3          | 0.3 | 0.3 | 0.07 | 0.3 | 10                | 4                        |
| 4          | 0.4 | 0.4 | 0.08 | 0.4 | 6                 | 2                        |
| 5          | 0.1 | 0.5 | 0.09 | 0.5 | 8                 | 3                        |
| 6          | 0.2 | 0.6 | 0.05 | 0.6 | 10                | 4                        |
| 7          | 0.3 | 0.1 | 0.06 | 0.1 | 6                 | 2                        |
| 8          | 0.4 | 0.2 | 0.07 | 0.2 | 8                 | 3                        |
| 9          | 0.1 | 0.3 | 0.08 | 0.3 | 10                | 4                        |
| 10         | 0.2 | 0.4 | 0.09 | 0.4 | 6                 | 2                        |
| 11         | 0.3 | 0.5 | 0.05 | 0.5 | 8                 | 3                        |

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd

# 1. Генерация обучающих и тестовых данных
def target_function(x, a=0.3, b=0.1, c=0.06, d=0.1):
    return a * torch.cos(b * x) + c * torch.sin(d * x)

a, b, c, d = 0.3, 0.1, 0.06, 0.1
input_size = 1
hidden_size = 2
window_size = 6
num_samples = 200
train_ratio = 0.8

x = torch.linspace(0, 100, num_samples).reshape(-1, 1)
y = target_function(x, a, b, c, d)

X = []
y_target = []
for i in range(num_samples - window_size):
    X.append(y[i:i+window_size])
    y_target.append(y[i:i+window_size])
X = torch.stack(X)
y_target = torch.stack(y_target)

split_idx = int(X.shape[0] * train_ratio)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y_target[:split_idx], y_target[split_idx:]
```

```
print("Задание 3: Данные сгенерированы для функции  $y = a \cdot \cos(bx) + c \cdot \sin(dx)$ ")
print(f"Размер обучающей выборки: {X_train.shape}, тестовой: {X_test.shape}")
```

*# 2. Архитектура РНС Элмана*

```
class ElmanRNN(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.rnn = nn.RNN(input_dim, hidden_dim, nonlinearity="tanh", batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        out, _ = self.rnn(x)
        out = out[:, -1, :]
        out = self.fc(out)
        return out
```

```
model = ElmanRNN(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

*# 3. Обучение модели*

```
losses = []
epochs = 5000

for epoch in range(epochs):
    model.train()
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
```

```
print("Обучение завершено. Минимальная ошибка:", min(losses))
```

*# 4. График ошибки по эпохам*

```
plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("График изменения ошибки по эпохам (РНС Элмана)")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()
```

*# 5. График прогнозируемой функции на обучающем участке*

```
model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)

plt.figure(figsize=(10, 4))
plt.plot(range(len(y_train)), y_train, label="Эталон")
plt.plot(range(len(y_train)), y_train_pred, label="Прогноз РНС Элмана")
plt.title("Прогнозируемая функция на обучающем участке")
plt.xlabel("Индекс")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

*# 6. Таблица результатов обучения*

```
train_table = pd.DataFrame({
    "Эталонное значение": y_train.squeeze().numpy(),
    "Полученное значение": y_train_pred.squeeze().numpy(),
    "Отклонение": (y_train_pred - y_train).squeeze().numpy()
})
```

```

print("\nРезультаты обучения (первые строки):")
print(train_table.head())

# 7. Таблица результатов прогнозирования
with torch.no_grad():
    y_test_pred = model(X_test)
    test_table = pd.DataFrame({
        "Эталонное значение": y_test.squeeze().numpy(),
        "Полученное значение": y_test_pred.squeeze().numpy(),
        "Отклонение": (y_test_pred - y_test).squeeze().numpy()
    })
print("\nРезультаты прогнозирования (первые строки):")
print(test_table.head())

```

## Результат:

Задание 3: Данные сгенерированы для функции  $y = a \cdot \cos(bx) + c \cdot \sin(dx)$

Размер обучающей выборки: `torch.Size([155, 6, 1])`, тестовой: `torch.Size([39, 6, 1])`

Обучение завершено. Минимальная ошибка: `1.4174034959069104e-06`

2025-12-02 10:56:45.743 Python[26224:950993] +[IMKClient subclass]: chose IMKClient\_Legacy

2025-12-02 10:56:45.743 Python[26224:950993] +[IMKInputSession subclass]: chose IMKInputSession\_Legacy

Результаты обучения (первые строки):

|   | Эталонное значение | Полученное значение | Отклонение |
|---|--------------------|---------------------|------------|
| 0 | 0.304285           | 0.304531            | 0.000246   |
| 1 | 0.302303           | 0.302728            | 0.000424   |
| 2 | 0.299559           | 0.300146            | 0.000587   |
| 3 | 0.296058           | 0.296789            | 0.000730   |
| 4 | 0.291810           | 0.292663            | 0.000853   |

Результаты прогнозирования (первые строки):

|   | Эталонное значение | Полученное значение | Отклонение |
|---|--------------------|---------------------|------------|
| 0 | -0.011952          | -0.012116           | -0.000165  |
| 1 | -0.027292          | -0.027253           | 0.000039   |
| 2 | -0.042564          | -0.042300           | 0.000265   |
| 3 | -0.057728          | -0.057224           | 0.000505   |
| 4 | -0.072747          | -0.071995           | 0.000752   |

Process finished with exit code 0



